PukSquash Developer Manual

Complete Firebase edition

4 October 2026

This manual brings together project setup and all twelve functionality sections. Each section includes the relevant source files, API behaviour, code explanations and verification guidance.

The implementation sections document Firebase Authentication and Firebase Data Connect at repository revision fe3e361. Registration payments are presented as a proposed Firebase extension.

Contents

00 Project Setup and Firebase Data Layer — 2

01 Registration Login and Access Control — 6

02 User Profile Management — 10

03 Court Availability Grid — 14

04 Court Booking Creation — 18

05 Joining an Open Booking — 22

06 My Bookings History and Cancellation — 26

07 Coach Availability and Notice Board — 30

08 Private Coaching Schedule — 34

09 Team Practice Bookings — 38

10 Administrator User Management — 42

11 Administrator Court Management — 46

12 Registration Payments and Firebase Extension — 50

PukSquash Developer Manual

Project Setup and Firebase Data Layer

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

PukSquash lets NWU Potchefstroom students check squash court availability and manage bookings remotely. This part explains the environment shared by the functionality manuals. The application uses Vue 3 and Vite for the browser, Laravel 12 and PHP for the REST API, Firebase Authentication for identity, and Firebase Data Connect with Cloud SQL PostgreSQL for relational data.

The browser sends a Firebase ID token to Laravel. Middleware verifies the identity, controllers apply feature rules, repositories call named GraphQL operations, and SqlConnectClient sends those operations to Data Connect. DTO classes translate the GraphQL response into the API field names consumed by Vue. Use revision fe3e361 when reproducing this edition; later repository code uses a different persistence implementation.

1 1 Files and Responsibilities

resources/js/pages and components — application screens and reusable interface elements

resources/js/services/http.js — authenticated JSON requests

app/Http/Controllers/Api and app/Repositories — feature logic and data access

app/Services/SqlConnectClient.php — Data Connect transport

dataconnect/schema/schema.gql — six core entities

dataconnect/example/queries.gql and mutations.gql — named operations

1 2 API Contract

All feature URLs in these manuals begin with /api. GET /api/me is the initial identity and profile check. Protected requests use Authorization: Bearer <Firebase ID token>.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Local development setup

Windows commands to run from the Firebase baseline checkout

```
composer install
copy .env.example .env
php artisan key:generate
npm ci
php artisan serve
# In a second terminal
npm run dev
```

Explanation

Use PHP 8.2 or a compatible version accepted by composer.lock, Composer, and a Node version supported by the locked Vite release. Install from the existing lockfiles. Copy the environment template only for a new local setup. Laravel serves the application; Vite provides development assets. Create the local Laravel framework database if the chosen cache/session drivers need it. Laravel framework migrations do not create the Data Connect domain schema.

Physical Code Explanation

1 composer install resolves the PHP libraries, including kreait/firebase-php, from composer.lock.

2 key:generate creates the Laravel application key. npm ci installs the JavaScript lockfile without selecting arbitrary newer versions.

3 php artisan serve starts the PHP application, while npm run dev starts Vite. npm run build produces production assets; neither command deploys the backend.

Set the six VITE_FIREBASE_* names read by resources/js/services/firebase.js. On the server, set FIREBASE_PROJECT_ID, FIREBASE_CREDENTIALS, FIREBASE_DATACONNECT_LOCATION, FIREBASE_DATACONNECT_SERVICE and FIREBASE_DATACONNECT_CONNECTOR. Credential paths are resolved relative to the repository by this implementation. Keep the service-account JSON outside the public directory and source control.

3 Data Connect request implementation

app/Services/SqlConnectClient.php | source line 69 onward; blank and comment lines omitted

```
public function execute(string $operationName, array $variables = []): array
{
    $query = $this->loadOperation($operationName);
    $request = Http::timeout(15);
    if (! $this->emulatorHost) {
        $request = $request->withToken($this->getAccessToken());
    }
    $response = $request->post($this->endpoint, [
        'query'         => $query,
        'operationName' => $operationName,
        'variables'     => (object) $variables,
    ]);
    if ($response->failed()) {
        throw new RuntimeException(
            "SQL Connect request [{$operationName}] failed: " . $response->body(),
            $response->status(),
        );
    }
    $body = $response->json();
```

Explanation

Each repository selects an operation by its exact GraphQL name and supplies a variables array. The service loads the definition from the connector files. A server OAuth access token authorises the request in the hosted environment; DATA_CONNECT_EMULATOR_HOST selects the local emulator and skips that token path.

Physical Code Explanation

1 loadOperation locates the named query or mutation and caches its text. A missing operation name throws an exception.

2 Http::timeout(15) bounds the external request. withToken supplies the server access token; it is separate from the browser Firebase ID token.

3 The request posts query, operationName and variables. An empty variables array is cast to an object so GraphQL receives the expected JSON shape.

User is linked to Firebase through firebaseUid. Court and TimeSlot describe bookable cells; Booking reserves a court/date/slot; BookingPlayer records participants; CoachAvailability has a composite coach/slot/date key. Connector operations use NO_ACCESS for privileged server access. Firestore collections and Storage rules from the example manual do not describe this relational data model.

4 Verification and Maintenance

Verification checklist

1 Run npm run build and php artisan test in the configured Firebase checkout. Record the actual output; these manuals do not claim a test run has passed.

2 Confirm GET /api/me works with a valid token and returns 401 without one. Confirm a Data Connect query reaches the intended project and service.

3 Use the Firebase CLI and the repository dataconnect configuration to prepare the emulator or deploy the schema and connector in a development project before seeding courts and slots. Review schema changes before deployment.

4 Check every one-hour TimeSlot and court before testing bookings. Keep production credentials and production data out of local tests.

Implementation boundaries

The checked firebase.json configures Data Connect and its emulator. It has no complete hosting configuration for the Laravel PHP backend. Provision a PHP-capable HTTPS host and deploy Vite assets with Laravel; do not assume firebase deploy hosts the whole application.

The specifications require transaction-safe reservations, booking limits and additional Phase 2 tools. Comments about uniqueness are not proof that a PostgreSQL constraint exists. Validate deployed constraints and add integration tests. Time-slot administration and configurable rules have no complete admin API in this baseline.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

https://firebase.google.com/docs/sql-connect/admin-sdk

PukSquash Developer Manual

Registration Login and Access Control

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

This functionality creates a Firebase email and password account, sends email verification, signs the user in and supplies verified identity to Laravel. The Firebase account owns the credentials. The PukSquash User record owns the application name, student number, role and account status.

SignUpPage calls registerUser and waits for verification. It reloads the Firebase user every three seconds and stops the timer when the component unmounts. GET /api/me creates the minimal Data Connect profile if it is missing. LoginPage signs in, checks emailVerified and refreshes the account. Vue route guards guide navigation; Laravel verifies protected requests independently.

1 1 Files and Responsibilities

resources/js/pages/SignUpPage.vue and LoginPage.vue — forms and verification flow

resources/js/features/auth/authService.js — registration, login and logout

resources/js/composables/useAuth.js and useAccount.js — shared identity and profile state

app/Services/FirebaseAuthService.php — Firebase server client

app/Http/Middleware/FirebaseAuth.php and AdminMiddleware.php — access checks

app/Http/Controllers/Api/UserController.php — GET /api/me

1 2 API Contract

Firebase SDK calls handle registration and login. GET /api/me reads or creates the application profile. There is no Laravel password-login endpoint in this edition.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Firebase registration implementation

resources/js/features/auth/authService.js | source line 49 onward; blank and comment lines omitted

```
export async function registerUser(email, password) {
    startLoading();
    try {
        const result = await createUserWithEmailAndPassword(
            auth,
            normalizeEmail(email),
            password
        );
        await sendEmailVerification(result.user);
        return result;
    } catch (error) {
        throw new Error(friendlyAuthError(error));
    } finally {
        stopLoading();
    }
}
```

Explanation

The authentication service uses the Firebase auth instance imported from services/firebase.js, creates the account and sends the verification email. It converts common Firebase error codes to readable messages. The shared loading indicator starts before the request and stops in finally, including on failure. Account creation and email verification are separate steps.

Physical Code Explanation

1 The auth instance is statically imported in this revision. startLoading begins shared feedback, and stopLoading in finally resets it. Configure Firebase before loading the authentication pages.

2 createUserWithEmailAndPassword receives the trimmed email and original password. Passwords are handled by Firebase, not stored in the Data Connect User entity.

3 sendEmailVerification sends the verification link. If sending fails after account creation, the account may already exist; retry and recovery must account for that partial outcome.

4 loginUser uses signInWithEmailAndPassword. logoutUser delegates to Firebase signOut. useAuth observes onAuthStateChanged for application-wide session state.

The signup form prevents repeated submission while waiting for verification. A verified signup proceeds to the booking page. The baseline has no registration payment gate. Payment-dependent registration is covered separately as an extension.

3 Profile creation after authentication

app/Http/Controllers/Api/UserController.php | source line 23 onward; blank and comment lines omitted

```
public function me(Request $request): JsonResponse
{
    $firebaseUid = $request->attributes->get('firebase_uid');
    $email       = $request->attributes->get('firebase_email');
    $user = $this->users->findByFirebaseUid($firebaseUid);
    if (! $user) {
        $studentNumber = $email ? explode('@', $email)[0] : null;
        $this->users->upsert([
            'firebaseUid' => $firebaseUid,
            'fullName'    => explode('@', $email)[0],
            'email'       => $email,
            'studentNumber' => $studentNumber,
        ]);
        $user = $this->users->findByFirebaseUid($firebaseUid);
    }
    return response()->json($user?->toArray());
}
```

Explanation

FirebaseAuth verifies the bearer token, resolves its subject UID and fetches the Firebase user. It places the UID, email and verification flag in request attributes. GET /api/me is deliberately allowed before email verification so the minimal profile can be created; other protected API requests reject an unverified account.

Physical Code Explanation

1 The controller reads firebase_uid and firebase_email from middleware attributes, not from browser input.

2 findByFirebaseUid links the Firebase identity to its application profile. If absent, the email prefix supplies a temporary display name and student number.

3 UserRepository::upsert defaults role to USER and status to ACTIVE, then executes the UpsertUser insert mutation. The controller reads the created record and returns its DTO array.

4 AdminMiddleware additionally requires an ACTIVE profile with role ADMIN. Coach controllers explicitly require an ACTIVE COACH. Client-side requiresRole metadata controls the screen, not the authority to call the API.

The method named UpsertUser is implemented with user_insert and duplicate-error handling in this baseline. It is not a general update-on-conflict mutation. Keep retries and cache invalidation consistent with that behaviour.

4 Verification and Maintenance

Verification checklist

1 Register a new test address: expect a Firebase account and verification email; verify the link and confirm navigation and GET /api/me.

2 Try a duplicate email, invalid email, weak password, incorrect password and unavailable network. Confirm an understandable error and a usable retry path.

3 Call a protected endpoint without a token or with an invalid token: expect 401. Call with an unverified account: expect 403, except GET /api/me.

4 Use USER, COACH and ADMIN test accounts against the actual protected APIs. A hidden link or blocked browser route alone is not an authorisation test.

5 Navigate away during verification and check that polling stops. Log out, then attempt to reuse the authenticated page and confirm that a new protected request requires authentication.

Implementation boundaries

The specifications require university email eligibility. The baseline trims email but does not implement an NWU domain allowlist. Add a server-enforced eligibility policy before claiming NWU-only registration.

FirebaseAuth in this baseline does not globally reject suspended or inactive application profiles; some role-specific controllers do. This is a known access-control gap for general booking routes. Password-reset and Google sign-in screens from the example manual are not implemented here.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

https://firebase.google.com/docs/auth/web/password-auth

https://firebase.google.com/docs/auth/admin/verify-id-tokens

PukSquash Developer Manual

User Profile Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Profile management lets a signed-in user edit the name shown in the application and on bookings. The Firebase edition keeps authentication credentials separate from the relational profile. User, Coach and Admin accounts use the same profile page.

UserProfile loads refreshAccount on mount, splits full_name into first and remaining names, and displays account details. Edit enables the name fields. Save opens a confirmation dialog, updateAccount sends PATCH /api/me, and the successful response replaces the shared profile. Cancel restores the last saved account values.

1 1 Files and Responsibilities

resources/js/pages/UserProfile.vue — editable form and confirmation

resources/js/composables/useAccount.js — shared profile and updateAccount

app/Http/Controllers/Api/UserController.php — update validation

app/Repositories/UserRepository.php — UpdateUserProfile and cache invalidation

1 2 API Contract

GET /api/me returns the profile. PATCH /api/me accepts full_name (required string, maximum 120 characters) and student_number (nullable string, maximum 30). The caller is identified from the verified Firebase token.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Profile update from the page

resources/js/pages/UserProfile.vue | source line 73 onward; blank and comment lines omitted

```
const saveProfile = async () => {
    error.value = '';
    confirmingSave.value = false;
    saving.value = true;
    try {
        const fullName = `${profile.value.firstName} ${profile.value.lastName}`.trim();
        await updateAccount({
            full_name: fullName,
            student_number: accountProfile.value?.student_number
                ?? accountProfile.value?.studentNumber
                ?? null,
        });
        applyFromAccount(accountProfile.value);
        isEditing.value = false;
        saved.value = true;
        setTimeout(() => { saved.value = false; }, 3000);
    } catch (e) {
        error.value = e?.message ?? String(e);
    } finally {
        saving.value = false;
    }
};
```

Explanation

The page combines the first and last name and sends the existing student number with the updated full_name. It waits for the account service before displaying success. saving disables repeat submission and is reset in finally, including when the API fails.

Physical Code Explanation

1 The confirmation flag is cleared before saving and the prior error is removed.

2 fullName joins the two editable name fields and trims outer whitespace.

3 updateAccount receives only the fields permitted by this screen. applyFromAccount then reloads the returned shared profile.

4 The success message clears after three seconds. catch stores the error text; finally restores the button state.

Email, role and student number are displayed as account details. In this baseline the screen edits names, although the backend also accepts a student_number value. Phone-number and email-edit functionality belongs to later code, not this Firebase profile implementation.

3 Data Connect profile persistence

app/Repositories/UserRepository.php | source line 135 onward; blank and comment lines omitted

```
public function updateProfile(string $firebaseUid, string $fullName, ?string
$studentNumber): void
    {
        $keyFirebase = 'sqlconnect_user_firebase_uid_' . $firebaseUid;
        $existing = $this->findByFirebaseUid($firebaseUid);
        Cache::forget($keyFirebase);
        if ($existing) {
            Cache::forget('sqlconnect_user_id_' . $existing->id);
        }
        $this->client->execute('UpdateUserProfile', [
            'firebaseUid'    => $firebaseUid,
            'fullName'       => $fullName,
            'studentNumber'  => $studentNumber,
        ]);
        Cache::forget($keyFirebase);
        if ($existing) {
            Cache::forget('sqlconnect_user_id_' . $existing->id);
        }
    }
}
```

Explanation

The repository resolves the current user, invalidates UID and application-ID cache entries, executes UpdateUserProfile, and invalidates again. The mutation updates fullName and studentNumber for the verified firebaseUid. The controller rereads the record so the browser receives persisted values.

Physical Code Explanation

1 The UID-specific cache key is built from the verified identity; the application ID cache is also cleared if a record exists.

2 execute passes camelCase variables to GraphQL even though the REST payload uses snake_case. This mapping is deliberate.

3 UpdateUserProfile changes only the profile fields in its mutation. It does not update Firebase Authentication credentials or promote a role.

When adding a profile field, change the schema, query selections, mutation variables, DTO conversion, request validation and Vue form together. Merely adding an input cannot make the field persist.

4 Verification and Maintenance

Verification checklist

1 Change the name, confirm Save, refresh the page and verify the value in the API response and in a booking label.

2 Cancel editing and confirm the original name is restored without a request.

3 Send a missing or over-length full_name: expect Laravel validation failure with HTTP 422.

4 Attempt to send another firebase_uid or role in the request: the update must remain scoped to the authenticated user and must not change privileges.

5 Simulate API failure and confirm the form remains recoverable and saving returns to false.

Implementation boundaries

The screen treats student number as read-only while the API permits its update. Decide whether it should be immutable and enforce that decision on the backend.

A new display name can appear in several cached or already-loaded views. Refresh the shared account and any affected booking data after changes; do not treat stale browser labels as a failed database update.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Court Availability Grid

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

The availability grid shows the state of every court and fixed time slot for a selected date. It provides the entry point for creating a booking or joining an Open booking. Laravel returns both the display state and action flags so the browser uses one set of availability rules.

BookCourtPage requests the selected date through bookingService. AvailabilityRepository fetches courts, active slots and confirmed bookings in one Data Connect operation. AvailabilityController orders the courts and slots, identifies past times and the current user's occupied times, then produces the cells rendered by AvailabilityGrid and explained by GridLegend.

1 1 Files and Responsibilities

resources/js/pages/BookCourtPage.vue — selected date and grid refresh

resources/js/components/AvailabilityGrid.vue and GridLegend.vue — display

resources/js/features/booking/bookingService.js — availability request

app/Http/Controllers/Api/AvailabilityController.php — state calculation

app/Repositories/AvailabilityRepository.php — GetAvailabilityData

1 2 API Contract

GET /api/availability?date=YYYY-MM-DD returns booking_window_days, courts, time_slots and cells. Each cell includes court_id, time_slot_id, state, label, can_book, can_join, is_mine and booking_id; occupied cells also include players.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Fetching the date snapshot

app/Repositories/AvailabilityRepository.php | source line 16 onward; blank and comment lines omitted

```
public function getForDate(string $bookingDate): array
    {
        $data = $this->client->execute('GetAvailabilityData', [
            'bookingDate' => $bookingDate,
        ]);
        $courts = array_map(
            fn (array $row) => CourtDTO::fromGraphQL($row),
            $data['courts'] ?? [],
        );
        $timeSlots = array_map(
            fn (array $row) => TimeSlotDTO::fromGraphQL($row),
            $data['timeSlots'] ?? [],
        );
        $bookings = array_map(
            fn (array $row) => BookingDTO::fromGraphQL($row),
            $data['bookings'] ?? [],
        );
        return [
            'courts'    => $courts,
            'timeSlots' => $timeSlots,
            'bookings'  => $bookings,
        ];
    }
}
```

Explanation

GetAvailabilityData returns the related data required to compute a complete grid. The repository maps the response into CourtDTO, TimeSlotDTO and BookingDTO objects. The controller then works with predictable property names instead of repeating raw GraphQL response handling.

Physical Code Explanation

1 The bookingDate variable selects the day. The operation name must match queries.gql exactly.

2 Each array_map converts a returned row to a typed DTO; missing response arrays default to empty arrays.

3 The returned snapshot keeps courts, timeSlots and bookings separate. Cells are derived by the controller, not persisted as a second availability table.

Times are interpreted in Africa/Johannesburg. The controller indexes confirmed bookings by court ID and slot ID, checks user participation, and disables free cells when the user is already booked in that time slot.

3 Choosing the occupied cell state

app/Http/Controllers/Api/AvailabilityController.php | source line 209 onward; blank and comment lines omitted

```
$state = match (true) {
    $booking->bookingType === 'PRIVATE_COACH'           => 'PRIVATE_COACH',
    $booking->bookingType === 'LOCKED'                  => 'LOCKED',
    $booking->bookingType === 'OPEN' && $playerCount < 2 => 'OPEN',
    default                                             => 'FULL',
};
$label = match ($state) {
    'OPEN'          => $nameA ? ($nameA . ' vs ?') : 'Open',
    'FULL'          => ($nameA && $nameB) ? ($nameA . ' vs ' . $nameB) : ($nameA ?:
'Full'),
    'LOCKED'        => $nameA ?: 'Locked',
    'PRIVATE_COACH' => ($nameA && $coachName)
        ? ($nameA . ' / Coach ' . $coachName)
        : ($nameA ?: ($coachName ? ('Coach ' . $coachName) : 'Coach')),
    default         => $nameA ?: $state,
};
```

Explanation

An existing booking determines the occupied state. A Private Coach or Locked booking remains closed to joining. An Open booking with fewer than two players is joinable only if the current caller is not already involved, the time has not started and the caller is free.

Physical Code Explanation

1 The match expression gives PRIVATE_COACH and LOCKED priority. OPEN is returned only for an Open booking with fewer than two players; other occupied cases become FULL.

2 The label uses participant and coach names when available and falls back to a readable state label.

3 Later in the method, canJoin also checks isMine, slotIsPast and userBusy. booking_id is provided for an eligible join; otherwise it is null.

A grid snapshot can become stale immediately after it is read. Recheck in the create and join endpoints. After a mutation or HTTP 409 conflict, refresh the selected date instead of silently showing the earlier grid.

4 Verification and Maintenance

Verification checklist

1 Use fixtures for AVAILABLE, OPEN, FULL, LOCKED, PRIVATE_COACH and UNAVAILABLE. Check both the legend colour and the action enabled on the cell.

2 Compare Court 2 and Court 10 ordering and check TimeSlot displayOrder.

3 Check today immediately before and after a slot starts, using South African time. Past cells must not offer booking or joining.

4 Book one court at a time slot, then check the other courts for that same user. Confirm overlapping participation is disabled.

5 Use two browser sessions: mutate a booking in one, refresh the other and verify the updated state.

Implementation boundaries

The payload advertises a fourteen-day booking window, but that field alone is not server enforcement of the window. The ordinary booking controller needs its own boundary check.

Live availability in this baseline means refreshed API snapshots. No WebSocket or Firestore realtime subscription is part of this feature.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Court Booking Creation

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

This functionality reserves one court for one fixed time slot on one date. OPEN creates a session another user can join. LOCKED reserves the cell without a join action. PRIVATE_COACH also references a coach who has published availability for that date and slot.

The user selects an available cell in BookCourtPage and completes BookingDialog. bookingService posts the selected identifiers and booking type. Laravel resolves the creator from the Firebase identity, validates the request, checks the court and time, checks conflicting user participation and court occupancy, and validates any selected coach. It creates the booking and then the creator's Player 1 record.

1 1 Files and Responsibilities

resources/js/components/BookingDialog.vue — booking type and coach selection

resources/js/features/booking/bookingService.js — createBooking and availableCoaches

app/Http/Controllers/Api/BookingController.php — store

app/Repositories/BookingRepository.php and BookingPlayerRepository.php — mutations

dataconnect/example/mutations.gql — CreateBooking and AddBookingPlayer

1 2 API Contract

POST /api/bookings accepts court_id, time_slot_id, booking_date, booking_type and optional coach_id. PRIVATE_COACH requires coach_id. Success is HTTP 201 with {"id":"<booking ID>"}. The caller cannot choose created_by_user_id.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validation and identity resolution

app/Http/Controllers/Api/BookingController.php | source line 31 onward; blank and comment lines omitted

```
public function store(Request $request): JsonResponse
{
    $request->validate([
        'court_id'     => 'required|string',
        'time_slot_id' => 'required|string',
        'booking_date' => 'required|date_format:Y-m-d',
        'booking_type' => 'required|in:OPEN,LOCKED,PRIVATE_COACH',
        'coach_id'     => 'nullable|string',
    ]);
    $firebaseUid = $request->attributes->get('firebase_uid');
    $user = $this->users->findByFirebaseUid($firebaseUid);
```

Explanation

The controller validates the external REST fields before doing the domain checks. All three booking types share the endpoint. The Firebase UID comes from middleware and resolves the application user, ensuring the reservation belongs to the authenticated caller.

Physical Code Explanation

1 The request validation fixes the date format and limits booking_type to OPEN, LOCKED or PRIVATE_COACH.

2 coach_id is nullable at initial validation because it is irrelevant to normal bookings. A later branch requires it for PRIVATE_COACH.

3 The controller reads firebase_uid from request attributes and queries the application user. It does not trust a user ID submitted by the client.

The remaining checks return 422 for an inactive/missing court, a past or current slot, or overlapping user participation; SLOT_TAKEN and COACH_UNAVAILABLE use 409. They must run again even when the browser offered the cell as available.

3 Creating the relational booking

dataconnect/example/mutations.gql | source line 159 onward; blank and comment lines omitted

```
mutation CreateBooking(
  $courtId: UUID!
  $timeSlotId: UUID!
  $createdById: UUID!
  $coachId: UUID
  $bookingDate: Date!
  $bookingType: String!
  $bookedAt: Timestamp!
)
@auth(level: NO_ACCESS) {
  booking_insert(
    data: {
      courtId: $courtId
      timeSlotId: $timeSlotId
      createdById: $createdById
      coachId: $coachId
      bookingDate: $bookingDate
      bookingType: $bookingType
      status: "CONFIRMED"
      bookedAt: $bookedAt
    }
  )
}
```

Explanation

The repository passes the controller's camelCase variables to CreateBooking. The mutation inserts a CONFIRMED Booking linked to the Court, TimeSlot, creator and optional coach. The response key is used for a separate AddBookingPlayer call with player position 1.

Physical Code Explanation

1 UUID variables identify the related entities, while bookingDate is a GraphQL Date and bookedAt is a Timestamp.

2 createdById is derived from the verified user. The stored status is explicitly CONFIRMED.

3 NO_ACCESS keeps ordinary client operation execution closed; Laravel uses privileged server access.

4 The controller accepts several insertion-key response shapes, extracts id, then adds Player 1. A missing key becomes HTTP 500.

This excerpt is the Firebase baseline implementation. The two inserts are separate external calls, and the preceding slot check is also separate. Do not describe them as a single transaction or a demonstrated concurrency guarantee.

4 Verification and Maintenance

Verification checklist

1 Create OPEN and LOCKED bookings in different free cells. Verify the Booking row and Player 1 relation, then refresh the grid.

2 Attempt the same court/date/slot again: expect 409 SLOT_TAKEN when the existing booking is observed.

3 Attempt a second booking at the same date/slot on another court: expect 422 USER_ALREADY_BOOKED.

4 Create PRIVATE_COACH without a coach and with an unavailable coach: expect 422 and 409 respectively.

5 Run concurrent requests and simulate a failure between Booking and Player 1 creation. Inspect for duplicate cells or orphan bookings; these are required integrity tests, not proven passing behaviour.

Implementation boundaries

The supplied specification requires normal booking limits, consecutive-booking limits and transactional conflict protection. The baseline store method does not fully implement those limits or one atomic reservation operation.

The schema contains comments describing composite booking and player uniqueness but does not declare all those constraints. Implement and verify a transaction-safe operation and database constraints before relying on concurrent booking safety.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Joining an Open Booking

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Joining adds the authenticated user as Player 2 to an existing OPEN booking. It reuses the same reservation, court, date and time slot. It must never create a second Booking for the same cell.

The availability response exposes a join action only for eligible Open cells. The browser passes the existing booking ID to joinBooking. Laravel checks the user and booking, then checks status, type, time, the user's other participation, capacity and duplicate membership. It inserts one BookingPlayer relation at position 2 and returns the original booking ID.

1 1 Files and Responsibilities

resources/js/features/booking/bookingService.js — joinBooking

resources/js/pages/BookCourtPage.vue — join action and refresh

app/Http/Controllers/Api/BookingController.php — join

app/Repositories/BookingPlayerRepository.php — add

dataconnect/example/mutations.gql — AddBookingPlayer

1 2 API Contract

POST /api/bookings/{id}/join has no user-ID payload. Success returns HTTP 200 and the existing id. Missing user/booking returns 404; invalid type, time or participation returns 422; an observed full booking returns 409 SLOT_TAKEN.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Eligibility and capacity checks

app/Http/Controllers/Api/BookingController.php | source line 204 onward; blank and comment lines omitted

```
    if (count($booking->players) >= 2) {
        return response()->json([
            'message' => 'This booking is already full.',
            'code'    => 'SLOT_TAKEN',
        ], 409);
    }
    foreach ($booking->players as $p) {
        if (($p->user['id'] ?? null) === $user->id) {
            return response()->json(['message' => 'You are already in this booking.'],
422);
        }
    }
    $this->players->add($id, $user->id, 2);
    return response()->json(['id' => $id]);
}
```

Explanation

After verifying that the booking is confirmed and Open and that the time is still eligible, this part rejects a full booking and duplicate participation. The caller is added only when both checks pass. The response identifies the booking that was joined.

Physical Code Explanation

1 count($booking->players) >= 2 rejects an already full booking with HTTP 409.

2 The participant loop compares application user IDs, preventing the same user from occupying both positions.

3 players->add receives the existing booking ID, the verified user ID and player position 2. No CreateBooking mutation is called.

4 The final response returns the same ID so the frontend can refresh the affected booking and date.

The controller also rejects a user who already participates in any booking at that date and time slot. Treat 409 as a normal contention result: tell the user the place is unavailable and refresh the grid.

3 Persisting Player 2

dataconnect/example/mutations.gql | source line 202 onward; blank and comment lines omitted

```
mutation AddBookingPlayer(
  $bookingId: UUID!
  $userId: UUID!
  $playerPosition: Int!
  $joinedAt: Timestamp!
)
@auth(level: NO_ACCESS) {
  bookingPlayer_insert(
    data: {
      bookingId: $bookingId
      userId: $userId
      playerPosition: $playerPosition
      joinedAt: $joinedAt
    }
  )
}
```

Explanation

AddBookingPlayer inserts the relationship between the reservation and the participant. It stores the player position and join timestamp. The position is selected on the server rather than accepted from the browser.

Physical Code Explanation

1 bookingId and userId are foreign-key identifiers. playerPosition is the server-selected 1 or 2 convention.

2 joinedAt records when the participant joined. The mutation itself is a privileged server operation.

3 The standalone mutation does not read the current capacity and write the new participant atomically. The preceding controller check cannot eliminate a simultaneous Player 2 race.

Use uniqueness on the booking/position and booking/user pairs and perform eligibility plus insertion in a transaction-safe operation. Repeated or concurrent requests should return a controlled conflict without overwriting a participant.

4 Verification and Maintenance

Verification checklist

1 Join an Open booking owned by another account. Verify one new BookingPlayer at position 2 and no additional Booking row.

2 Try to join LOCKED, PRIVATE_COACH and cancelled bookings: expect rejection.

3 Try joining as the creator or an existing player. Confirm there is no second membership for the same user.

4 Try to join after the slot starts and while already booked at the same time on another court.

5 Submit two Player 2 requests concurrently. Verify one successful membership and one controlled failure after the required database protection is implemented.

Implementation boundaries

Composite uniqueness is described in schema comments but is not fully declared in the baseline BookingPlayer type. A capacity check followed by a separate insert is a known implementation gap.

When Player 2 later leaves, remove only that participant relationship. The creator’s Booking and Player 1 record continue to reserve the court.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

My Bookings History and Cancellation

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

My Bookings displays the authenticated user’s reservations and Open sessions they joined. It supports upcoming, past and cancelled scopes. The cancellation endpoint distinguishes cancelling the creator’s reservation from leaving another user’s Open session.

MyBookingsPage requests a scope and renders BookingCard components. The repository returns reservations where the user is the creator or a participant. The controller applies date/time filtering and sorts the result. When the user confirms cancellation or leaving, the endpoint derives the caller from Firebase and checks ownership or participation before changing data.

1 1 Files and Responsibilities

resources/js/pages/MyBookingsPage.vue and components/BookingCard.vue — list and actions

resources/js/components/ConfirmDialog.vue — confirmation pattern

resources/js/features/booking/bookingService.js — myBookings and cancelBooking

app/Http/Controllers/Api/BookingController.php — mine and cancel

app/Repositories/BookingRepository.php and BookingPlayerRepository.php — cancellation and leaving

1 2 API Contract

GET /api/bookings/mine?scope=upcoming|past|cancelled returns an array. POST /api/bookings/{id}/cancel returns {"left":false} for creator cancellation or {"left":true} for leaving an Open session.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Creator and participant behaviour

app/Http/Controllers/Api/BookingController.php | source line 352 onward; blank and comment lines omitted

```
    $isCreator = ($booking->createdBy['id'] ?? null) === $user->id;
    if (! $isCreator) {
        if ($booking->bookingType !== 'OPEN') {
            return response()->json(['message' => 'You are not authorised to cancel this
booking.'], 403);
        }
        $isPlayer = false;
        foreach ($booking->players as $p) {
            if (($p->user['id'] ?? null) === $user->id) {
                $isPlayer = true;
                break;
            }
        }
        if (! $isPlayer) {
            return response()->json(['message' => 'You are not authorised to cancel this
booking.'], 403);
        }
        $this->players->remove($id, $user->id);
        return response()->json(['left' => true]);
    }
    $this->bookings->cancel($id);
    return response()->json(['left' => false]);
}
```

Explanation

Only the creator can cancel the reservation itself. A non-creator may leave only if the booking is Open and the user is one of its participants. A caller with no relationship to the booking is denied. This distinction preserves the original reservation when Player 2 leaves.

Physical Code Explanation

1 isCreator compares the booking creator ID with the verified application user ID.

2 A non-creator is rejected with 403 unless the type is OPEN and their membership is found.

3 players->remove deletes that user's participation and returns left=true.

4 For the creator, bookings->cancel marks the reservation cancelled and returns left=false. Earlier checks reject missing or already cancelled bookings.

The list uses Africa/Johannesburg time. In the baseline, upcoming/past classification is based on the slot start, while the coach schedule keeps sessions until their end. That difference is intentional to document rather than silently normalise.

3 Cancellation and leaving mutations

app/Repositories/BookingRepository.php | source line 93 onward; blank and comment lines omitted

```
public function cancel(string $id): void
{
    $this->client->execute('CancelBooking', [
        'id'          => $id,
        'cancelledAt' => now()->toIso8601ZuluString(),
    ]);
}
```

Explanation

Cancellation changes the Booking status to CANCELLED and sets cancelledAt through the CancelBooking mutation. Availability queries consider confirmed bookings, so the cancelled reservation no longer blocks the court. Leaving an Open booking instead calls the LeaveBooking mutation, which deletes the participant relation matching both booking ID and user ID.

Physical Code Explanation

1 execute selects CancelBooking and provides the booking key plus the server's cancellation timestamp.

2 The GraphQL mutation updates the status rather than deleting the booking, retaining a cancelled record for history.

3 LeaveBooking uses a constrained deleteMany matching bookingId and userId; it does not remove the creator or other participants.

My Bookings fetches confirmed or cancelled data and then filters by creator or participant. The baseline repository fetches all bookings of the selected status before filtering in PHP; a server-side user filter and pagination would improve scalability.

4 Verification and Maintenance

Verification checklist

1 Confirm a creator appears in My Bookings and Player 2 also sees the joined Open booking.

2 As Player 2, leave: expect left=true, the creator still listed and the grid Open again.

3 As the creator, cancel: expect left=false, a cancelled history record and a free cell after refresh.

4 Try cancelling another user's Locked booking: expect 403. Try cancelling an already cancelled booking: expect 422.

5 Check today's boundaries for upcoming and past, and verify cancelled entries are selected by status.

Implementation boundaries

The cancel method does not enforce a future-time cutoff in this baseline. A disabled frontend action is not a backend cancellation policy; add the agreed rule explicitly if required.

Unsupported scope values are not explicitly validated in mine. Validate the accepted scopes before extending the endpoint, and add tests for unexpected values.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Coach Availability and Notice Board

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Coaches publish the fixed time slots during which they can offer private coaching. Students browse those published times through the coach notice board. Availability describes a coach’s offer; it does not itself reserve a court.

An ACTIVE COACH submits a date and one or more time-slot IDs. The request validator rejects malformed or duplicate IDs and past dates. The controller checks active slots and times, skips availability already published, and creates missing rows. The coach can retrieve their published slots and remove an eligible slot. The notice board groups valid rows by coach and sorts slots for display.

1 1 Files and Responsibilities

resources/js/pages/CoachBookings.vue — availability management

resources/js/pages/CoachNoticeBoardPage.vue — coach notice board

app/Http/Requests/PublishCoachAvailabilityRequest.php — input rules

app/Http/Controllers/Api/CoachAvailabilityController.php — publish, mine, destroy and noticeBoard

app/Repositories/CoachAvailabilityRepository.php — named Data Connect operations

1 2 API Contract

PUT /api/coach/availability accepts availability_date and time_slot_ids. GET and DELETE /api/coach/availability manage the caller’s slots. GET /api/notice-board/coaches?date=YYYY-MM-DD lists coaches. GET /api/coaches/available also takes time_slot_id.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validating published availability

app/Http/Requests/PublishCoachAvailabilityRequest.php | source line 26 onward; blank and comment lines omitted

```
public function rules(): array
    {
        return [
            'availability_date' => [
                'required',
                'date_format:Y-m-d',
                'after_or_equal:today',
            ],
            'time_slot_ids' => [
                'required',
                'array',
                'min:1',
            ],
            'time_slot_ids.*' => [
                'required',
                'string',
                'regex:/\A[0-9a-fA-F]{32}\z/',
                'distinct',
            ],
        ];
    }
}
```

Explanation

The FormRequest validates the date and selected slot identifiers before the controller writes anything. Authorisation remains in Firebase middleware and the controller's ACTIVE COACH check; authorize returning true does not grant every user coaching privileges.

Physical Code Explanation

1 availability_date must use YYYY-MM-DD and cannot be before today.

2 time_slot_ids must contain at least one item.

3 Each item must be a distinct 32-character hexadecimal string in this baseline. Do not substitute a hyphenated UUID in a test without matching the actual validator.

4 The controller separately checks that every requested ID belongs to an active TimeSlot and that today's selected times have not started.

Publishing returns created_time_slot_ids and existing_time_slot_ids. Repeating the same selection should not create duplicates. The controller batches missing rows through createMany and treats recognised duplicate errors as existing availability.

3 Composite availability identity

dataconnect/schema/schema.gql | source line 173 onward; blank and comment lines omitted

```
type CoachAvailability @table(key: ["coach", "timeSlot", "availabilityDate"]) {
  coach: User!
  timeSlot: TimeSlot!
  availabilityDate: Date!
  createdAt: Timestamp! @default(expr: "request.time")
}
```

Explanation

A CoachAvailability row is identified by coach, timeSlot and availabilityDate. That composite identity reflects the rule that one coach publishes a given date/slot once. Unlike ordinary Bookings, an availability row has no court relation.

Physical Code Explanation

1 The @table key declares the composite identity in the schema.

2 coach points to a User, so the server must enforce that the user is an ACTIVE COACH. A relation alone does not prove the role.

3 timeSlot points to the fixed slot shared with bookings. availabilityDate distinguishes the same hour on different days.

4 createdAt records when the availability was published.

The management response adds has_confirmed_booking. Removal checks for a confirmed coaching booking and returns 409 when one exists. The notice board filters out inactive coaches and inactive slots, deduplicates slots by ID and orders coaches by name. Published availability and actual bookability must still be distinguished at booking time.

4 Verification and Maintenance

Verification checklist

1 Publish valid slots as an ACTIVE COACH and verify both the management list and notice board. Repeat the request and check existing IDs are reported without duplicates.

2 Submit as USER or ADMIN: the active-coach endpoints must return 403.

3 Try duplicate, malformed, inactive and past slot selections. Verify rejection before new availability is created.

4 Remove an unbooked slot, then try removing one with a confirmed private session: expect success and 409 respectively.

5 Run PublishCoachAvailabilityTest, PublishCoachAvailabilityRequestTest and ManageCoachAvailabilityTest; separately verify the real Data Connect behaviour because mocked repositories do not test the deployed connector.

Implementation boundaries

Today comparisons in publishing use the Laravel application timezone, while several booking paths explicitly use Africa/Johannesburg. Configure and test the timezone consistently.

Batch publication uses separate operations and is not an all-or-nothing transaction. A non-duplicate failure can leave earlier successful rows published; handle retry and feedback accordingly.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Private Coaching Schedule

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

The coaching schedule shows confirmed private sessions assigned to the authenticated coach. It includes sessions that have not started and sessions still in progress, then removes them when their end time passes.

CoachBookings calls myCoachBookingsUpcoming through the booking service. Laravel resolves the caller’s profile, requires ACTIVE COACH, retrieves confirmed private-coaching reservations for that coach, filters by the session end time and sorts by date, start time and ID. The response contains the student, court, slot and status needed by the coach table.

1 1 Files and Responsibilities

resources/js/pages/CoachBookings.vue — schedule table

resources/js/features/booking/bookingService.js — myCoachBookingsUpcoming

app/Http/Controllers/Api/CoachBookingController.php — upcoming and toResponseArray

app/Repositories/BookingRepository.php — getConfirmedByCoach

dataconnect/example/queries.gql — GetConfirmedBookingsByCoach

1 2 API Contract

GET /api/coach/bookings/upcoming returns an array of id, player, booking_date, time_slot, court and status. The API determines the coach from the Firebase token; there is no caller-supplied coach ID.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Filtering current and future sessions

app/Http/Controllers/Api/CoachBookingController.php | source line 40 onward; blank and comment lines omitted

```
$now = CarbonImmutable::now('Africa/Johannesburg');
$bookings = $this->bookings->getConfirmedByCoach($user->id);
$bookings = array_values(array_filter(
    $bookings,
    function (BookingDTO $booking) use ($now): bool {
        $endTime = $booking->timeSlot['endTime'] ?? null;
        if (! $endTime) {
            return false;
        }
        $endsAt = CarbonImmutable::parse(
            $booking->bookingDate.' '.$endTime,
            'Africa/Johannesburg',
        );
        return $endsAt->greaterThan($now);
    },
));
```

Explanation

The schedule deliberately uses the end time. A coaching session remains visible during its full hour even after its start. Cancelled or differently assigned sessions are excluded by the repository query before this time filter is applied.

Physical Code Explanation

1 CarbonImmutable establishes the current time in Africa/Johannesburg.

2 getConfirmedByCoach selects the authenticated coach’s confirmed coaching reservations.

3 The filter reads endTime, combines it with bookingDate, and keeps the row only when endsAt is later than now.

4 A row without an end time is excluded. The array is reindexed so JSON remains a normal list.

The controller then sorts by booking date, start time and ID for deterministic ordering. The page receives only the fields needed for the schedule rather than a raw GraphQL response.

3 Retrieving the coach reservations

app/Repositories/BookingRepository.php | source line 143 onward; blank and comment lines omitted

```
public function getConfirmedByCoach(string $coachId): array
{
    $data = $this->client->execute('GetConfirmedBookingsByCoach', [
        'coachId' => $coachId,
    ]);
    return array_map(
        fn (array $row) => BookingDTO::fromGraphQL($row),
        $data['bookings'] ?? [],
    );
}
```

Explanation

The repository delegates selection to GetConfirmedBookingsByCoach and converts each result to BookingDTO. The controller maps these DTOs into the schedule response. Keeping this conversion in one place prevents GraphQL camelCase fields from leaking inconsistently into the Vue table.

Physical Code Explanation

1 coachId is the application User ID, not the Firebase UID. The middleware and UserRepository lookup establish the relationship.

2 execute sends the named query and its variables to Data Connect.

3 array_map constructs one BookingDTO per result; an empty result becomes an empty list.

Private coaching booking creation is documented in Court Booking Creation. This schedule does not publish coach availability, collect a coaching fee or create an additional reservation.

4 Verification and Maintenance

Verification checklist

1 Book a private session with the test coach and check the student name, court, date and time in the schedule.

2 Confirm the session is present before its start and while in progress, and absent at or after its end.

3 Use a different coach account and verify the session is not returned.

4 Verify USER, inactive COACH and unverified accounts cannot retrieve a coach schedule.

5 Run UpcomingCoachBookingsTest for the controller contract, then verify the actual connector query in a development Firebase project.

Implementation boundaries

A historical coaching endpoint is not implemented by this upcoming method. Add a distinct history contract if Phase 2 requires it rather than reusing an endpoint that filters past sessions out.

Private coaching payments are a specification extension. A confirmed booking in this Firebase baseline does not demonstrate that a fee was collected.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Team Practice Bookings

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Team practice lets an ACTIVE COACH select several court/date/time-slot cells and reserve them for squad training. Each selected cell becomes a Booking with bookingType TEAM_PRACTICE. The operation does not overwrite another reservation.

The team-practice page requests a dedicated availability grid and builds a selections array. Laravel validates the complete selection list, including duplicate cells, dates, active courts and slots and existing reservations. It prepares one booking row per cell and sends them through createMany. Successful booking IDs are returned to the page and can be retrieved for the coach’s selected date.

1 1 Files and Responsibilities

resources/js/pages/CoachTeamPracticeBookingPage.vue — multiple selection interface

resources/js/features/booking/bookingService.js — team-practice requests

app/Http/Controllers/Api/TeamPracticeController.php — availability, store and mine

app/Repositories/BookingRepository.php — createMany and team-practice query

1 2 API Contract

GET /api/coach/team-practice/availability?date=YYYY-MM-DD. POST /api/coach/team-practices accepts {selections:[{court_id,time_slot_id,booking_date}]}. GET /api/coach/team-practices?date=YYYY-MM-DD reads the coach’s sessions.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Preparing the selected reservations

app/Http/Controllers/Api/TeamPracticeController.php | source line 283 onward; blank and comment lines omitted

```
$bookedAt = now()->toIso8601ZuluString();
$rows = array_map(fn (array $sel) => [
    'courtId' => $sel['court_id'],
    'timeSlotId' => $sel['time_slot_id'],
    'createdById' => $user->id,
    'coachId' => null,
    'bookingDate' => $sel['booking_date'],
    'bookingType' => 'TEAM_PRACTICE',
    'bookedAt' => $bookedAt,
], $validated['selections']);
$results = $this->bookings->createMany($rows);
```

Explanation

After all selections pass validation, the controller maps REST field names to the GraphQL variables used for ordinary booking insertion. The same timestamp groups the submitted cells operationally, but the database stores a separate Booking for each one.

Physical Code Explanation

1 bookedAt is created once for the submission.

2 Each row contains the court, slot, booking date and the authenticated coach as createdById.

3 coachId is null because this is a team practice rather than a private lesson assigned through the coach relation.

4 bookingType is TEAM_PRACTICE. createMany sends the prepared rows to the repository.

The backend checks a fourteen-day window, rejects past or started cells, verifies active slots and courts, and rejects duplicate or occupied selections. The team route does not apply ordinary per-user overlapping or count checks, which supports multiple courts at the same time.

3 Batch insertion through Data Connect

app/Repositories/BookingRepository.php | source line 74 onward; blank and comment lines omitted

```
public function createMany(array $rows): array
{
    return $this->client->executeMany(
        'CreateBooking',
        array_map(
            fn (array $row) => [
                'courtId'     => $row['courtId'],
                'timeSlotId'  => $row['timeSlotId'],
                'createdById' => $row['createdById'],
                'coachId'     => $row['coachId'] ?? null,
                'bookingDate' => $row['bookingDate'],
                'bookingType' => $row['bookingType'],
                'bookedAt'    => $row['bookedAt'],
            ],
            $rows,
        ),
    );
}
```

Explanation

The repository maps each row and asks SqlConnectClient::executeMany to issue multiple CreateBooking requests. The client returns an ok flag and either data or an error for each request. This is parallel request execution, not a single PostgreSQL transaction.

Physical Code Explanation

1 The operation name remains CreateBooking, so the same core insertion contract is reused.

2 array_map ensures each request has exactly the required GraphQL variables. coachId falls back to null.

3 executeMany preserves result order for the caller to examine and returns successes and failures separately.

The controller attempts to cancel IDs already accumulated when it encounters a failure. This compensation can itself fail, and later requests may already have succeeded. A failed batch can therefore leave residual reservations. Replace this with a verified atomic operation before promising all-or-nothing booking.

4 Verification and Maintenance

Verification checklist

1 Book several free cells across courts and inspect the returned booking_ids and TEAM_PRACTICE rows.

2 Submit an occupied cell, inactive court, inactive slot, duplicate cell, past date and date beyond fourteen days; verify the specified 409 or 422 rejection.

3 Call the endpoints as a non-coach or inactive coach: expect 403.

4 Inject failure into the middle of a batch and inspect every selected cell, including later successes. Reconcile residual bookings rather than assuming rollback.

5 Run TeamPracticeBookingTest, then add real connector tests for concurrent batches and compensation failures.

Implementation boundaries

TEAM_PRACTICE is used by the implementation even though older schema comments enumerate only the three ordinary booking types. The field is a string, so documentation and validation must be maintained together.

This baseline does not create ordinary Player 1 rows for team-practice insertions. The creator identifies the responsible coach; avoid interpreting an empty participant list as a free court.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Administrator User Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

User management lets an ACTIVE ADMIN inspect application users, change role or status, and delete an account. The application role is held in Data Connect rather than inferred from a browser flag. Account deletion touches both Firebase Authentication and the relational profile.

ManageUsers loads the admin user list and submits selected changes. The routes run FirebaseAuth followed by AdminMiddleware. The controller validates allowed role and status values and delegates to UserRepository. Deletion first removes the Firebase Authentication account and then removes the Data Connect User record.

1 1 Files and Responsibilities

resources/js/pages/ManageUsersPage.vue — user list and edit actions

app/Http/Middleware/AdminMiddleware.php — ACTIVE ADMIN requirement

app/Http/Controllers/Api/AdminUserController.php — list, show, update and destroy

app/Repositories/UserRepository.php — Data Connect user operations

app/Services/FirebaseAuthService.php — Firebase account deletion

1 2 API Contract

GET /api/admin/users and GET /api/admin/users/{id} read users. PUT /api/admin/users/{id} accepts role and/or status. DELETE /api/admin/users/{id} removes the account. Allowed roles: USER, COACH, ADMIN. Allowed statuses: ACTIVE, INACTIVE, SUSPENDED.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validating an administrative update

app/Http/Controllers/Api/AdminUserController.php | source line 39 onward; blank and comment lines omitted

```
public function update(Request $request, string $id): JsonResponse
{
    $validated = $request->validate([
        'role'          => 'sometimes|in:USER,ADMIN,COACH',
        'status'        => 'sometimes|in:ACTIVE,INACTIVE,SUSPENDED',
    ]);
    if (! array_key_exists('role', $validated) && ! array_key_exists('status',
$validated)) {
        return response()->json(['message' => 'No changes provided.'], 422);
    }
    $this->users->updateUser($id, $validated);
    $user = $this->users->findById($id);
    return response()->json($user?->toArray());
}
```

Explanation

The update endpoint permits role and status changes but rejects an empty change request. It passes validated fields to the repository and returns the reread profile. AdminMiddleware must run before this method; the controller itself is not the complete access boundary.

Physical Code Explanation

1 sometimes allows changing one permitted field without sending the other. Each value is constrained to its known set.

2 The empty-change branch returns HTTP 422 instead of submitting a no-op update.

3 updateUser sends the update to Data Connect and invalidates cached user data.

4 findById retrieves the updated record for the response so the table can display server-authoritative values.

The baseline has no explicit last-administrator or self-demotion safeguard. Add an agreed policy and tests before treating such changes as operationally safe.

3 Updating Data Connect user fields

app/Repositories/UserRepository.php | source line 104 onward; blank and comment lines omitted

```
public function updateUser(string $id, array $params): void
{
    $existing = $this->findById($id);
    if ($existing) {
        $this->forgetCachedUser($existing);
    }
    $this->client->execute('UpdateUser', array_merge(
        ['id' => $id],
        $params,
    ));
    if ($existing) {
        $this->forgetCachedUser($existing);
    }
}
```

Explanation

The repository clears the current profile's caches before and after UpdateUser. The GraphQL mutation targets the application user ID and writes role/status. Clearing both the ID and Firebase UID caches prevents old privileges being retained merely because a cached profile was returned.

Physical Code Explanation

1 findById obtains the existing identity needed to invalidate both cache keys.

2 array_merge adds the target id to the validated update fields.

3 execute invokes UpdateUser using the server Data Connect transport.

4 The second invalidation prevents a stale lookup made during the update sequence from remaining cached.

destroy removes the Firebase user first; an already missing Firebase identity is tolerated. The subsequent relational deletion is separate. A failed relational delete can leave a profile whose Firebase identity is gone, so account deletion needs a recoverable operational procedure.

4 Verification and Maintenance

Verification checklist

1 Attempt every admin user route as USER and COACH: expect 403. Test an inactive ADMIN as well.

2 Change USER to COACH and refresh the account in the affected session. Confirm coach APIs check the updated server role.

3 Send an unknown role, unknown status or empty update: expect validation failure.

4 Delete a disposable account without related bookings and verify both identity and profile removal.

5 Test deletion with existing booking references and a simulated Firebase or Data Connect failure. Record and repair partial outcomes; do not use real accounts for this test.

Implementation boundaries

The global Firebase middleware in the baseline does not enforce application status for all ordinary endpoints. Changing status is persisted, but complete suspension enforcement requires a shared backend check.

The baseline deletion path does not first resolve booking references or provide a cross-service transaction. Prefer controlled deactivation where history must be retained and define the referential policy.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Administrator Court Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Court management maintains the physical courts offered in the availability grid. An ACTIVE ADMIN can create a court, change its name or description, change its ACTIVE/INACTIVE status, or request deletion. An inactive court must not be offered for new bookings.

ManageCourts calls the protected admin endpoints. Laravel validates submitted fields and uses CourtRepository to run the corresponding Data Connect operation. Changes are reread where appropriate, and the user-facing availability grid must be refreshed so new court status is visible.

1 1 Files and Responsibilities

resources/js/pages/ManageCourtsPage.vue — administrative court interface

app/Http/Controllers/Api/AdminCourtController.php — court CRUD endpoints

app/Repositories/CourtRepository.php — queries, mutations and cache handling

dataconnect/schema/schema.gql — Court type

dataconnect/example/mutations.gql — CreateCourt, UpdateCourt and DeleteCourt

1 2 API Contract

GET /api/admin/courts lists courts. POST creates one and returns 201. PUT /api/admin/courts/{id} updates an existing court. DELETE /api/admin/courts/{id} returns 204 on successful deletion. Public feature reads still use authenticated /api/courts endpoints.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Creating a court

app/Http/Controllers/Api/AdminCourtController.php | source line 25 onward; blank and comment lines omitted

```
public function store(Request $request): JsonResponse
{
    $validated = $request->validate([
        'name'     => 'required|string|max:255',
        'description' => 'sometimes|string|max:255',
        'status'   => 'sometimes|in:ACTIVE,INACTIVE',
    ]);
    $this->courts->create($validated);
    return response()->json(['message' => 'Court created successfully.'], 201);
}
```

Explanation

The controller accepts a required court name and optional description and status. Request validation restricts field lengths and the status vocabulary. CourtRepository provides defaults and executes CreateCourt; the endpoint responds with a success message rather than a complete court object.

Physical Code Explanation

1 name must be a string no longer than 255 characters.

2 description is optional but, when supplied to store, must be a string no longer than 255 characters.

3 status is optional and must be ACTIVE or INACTIVE.

4 The repository handles persistence; response()->json sends HTTP 201 so the frontend can reload the court list.

For an update, the controller verifies the court exists, validates optional fields, and chooses UpdateCourt or the status-only operation. Missing courts return 404 and requests with no recognised changes return 422.

3 Court schema and identity

dataconnect/schema/schema.gql | source line 57 onward; blank and comment lines omitted

```
type Court @table {
  name: String! @unique
  description: String
  status: String! @default(value: "ACTIVE")
  createdAt: Timestamp! @default(expr: "request.time")
  updatedAt: Timestamp! @default(expr: "request.time")
}
```

Explanation

The Court entity has a unique name, optional description and status defaulting to ACTIVE. Bookings reference its generated application key. Deactivation preserves those references while preventing new selection of unused inactive cells.

Physical Code Explanation

1 name is non-null and unique. A duplicate name is a database conflict, even if the form's string validation passes.

2 description is nullable in the schema. The update controller permits null, while the creation controller expects a string if supplied.

3 status is a string with an ACTIVE default. The allowed vocabulary is constrained by request validation, not an enum declaration in this schema.

4 createdAt and updatedAt have request-time defaults; update timestamps must be explicitly maintained where required by the chosen mutation.

Deleting a court can fail when reservations reference it. The baseline DeleteCourt path does not implement an archive workflow or a custom conflict response. Treat deactivation as the normal operational option for temporarily unavailable courts.

4 Verification and Maintenance

Verification checklist

1 Create a new court with and without optional fields. Confirm its appearance in the admin list and the availability grid.

2 Rename a court and verify natural ordering, including names containing 2 and 10.

3 Deactivate a court and confirm a fresh booking request for it returns 422 even if the browser had an older grid snapshot.

4 Try an unknown ID, over-length name, duplicate name and invalid status. Verify controlled UI feedback.

5 Delete a disposable court without bookings and confirm 204 is handled without JSON parsing. Test the referenced-court case separately.

Implementation boundaries

There is no complete administrator time-slot maintenance endpoint in this Firebase baseline. Court management must not be documented as providing time-slot CRUD.

The later admin dashboard statistics are not part of the documented Firebase baseline. Implement equivalent Data Connect queries before documenting live counts or utilisation as supported Firebase behaviour.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

PukSquash Developer Manual

Registration Payments and Firebase Extension

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

The functional and technical specifications introduce PayFast registration and private-coaching payments in Phase 2. This part describes the Firebase integration boundary and the work required to add those payments to the documented Firebase edition. It is an extension design, not a claim that revision fe3e361 already collects fees.

The later repository contains registration checkout and PayFast callback code. Its business flow can guide the extension: authenticate with Firebase, verify email, create a pending registration and payment attempt, send the user to hosted checkout, validate the server callback, then complete registration idempotently. The Data Connect schema and operations for this flow still need implementation.

1 1 Files and Responsibilities

resources/js/pages/RegistrationPaymentPage.vue, PaymentReturnPage.vue and PaymentCancelPage.vue — later checkout screens

app/Http/Controllers/Api/RegistrationController.php and PayfastItnController.php — later flow reference

app/Services/PayfastService.php and config/payfast.php — gateway logic

dataconnect/schema/schema.gql and connector operations — required Firebase extension

app/Http/Middleware/EnsureRegisteredUser.php — later registration gate to adapt

1 2 API Contract

Later reference routes are GET /api/registration/status, POST /api/registration/payment/initiate, POST /api/registration/reconcile and POST /api/payfast/itn/registration. These are not present in the fe3e361 route set. The ITN callback authenticates the payment notification rather than a browser Firebase token.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Proposed registration state handling

Proposed state flow for the Firebase extension

```
Firebase account created
        -> Email verification pending
Verified Firebase identity
        -> Pending registration
Payment attempt created on the server
        -> Hosted PayFast checkout
Verified COMPLETE notification
        -> Record payment as paid
        -> Create or activate application profile
        -> Mark registration completed
Duplicate notification
        -> Return the same completed result
```

Explanation

Keep identity creation separate from payment completion. A browser return URL is navigation, not payment proof. The backend must compare the callback with the stored payment attempt and validate the gateway notification before recording a successful payment or granting paid access.

Physical Code Explanation

1 Pending registration stores the verified Firebase UID and the profile fields waiting for activation. It must not contain the user's password.

2 A payment attempt has a unique merchant payment reference, server-selected amount in integer cents, currency, status and gateway reference.

3 A verified callback can arrive more than once or before the browser returns. Completion must produce one profile and one paid result.

4 If payment is verified but profile completion fails, retain enough durable state for reconciliation without charging again.

The fee is server configuration, not a browser-controlled amount. Keep PAYFAST_MERCHANT_ID, PAYFAST_MERCHANT_KEY and PAYFAST_PASSPHRASE server-side. Configure return, cancel and notify URLs for the correct environment.

3 Required Data Connect persistence contract

Design contract — names are proposed, not existing GraphQL operations

```
Required new entities
  PendingRegistration
  RegistrationPayment

Required server operations
  ReadRegistrationStatus
  CreateRegistrationPaymentAttempt
  CompleteRegistrationPayment
  ReconcileRegistration

Completion invariants
  Unique merchant payment reference
  Unique provider transaction reference where applicable
  Exact stored amount and currency match
  One application profile per Firebase UID
  Repeat completion has no duplicate side effects
```

Explanation

Add and review the new schema and privileged operations before connecting the later checkout screens to the Firebase edition. Payment completion needs a transaction-safe database operation. Independent calls to SqlConnectClient::execute do not become atomic simply because a Laravel DB::transaction surrounds them.

Physical Code Explanation

1 ReadRegistrationStatus must be scoped to the verified caller. A caller must not inspect another user's payment by supplying their UID.

2 CreateRegistrationPaymentAttempt records the expected fee before checkout so the callback can be checked against a server record.

3 CompleteRegistrationPayment validates the stored state and applies idempotent changes atomically within the persistence layer.

4 ReconcileRegistration safely retries incomplete activation after payment verification; it must not trust a browser assertion that payment succeeded.

Check PayFast signature, merchant, expected amount, allowed payment status and server confirmation, with source validation configured correctly for the deployment. Production must not enable relaxed validation or demo-completion routes. See the official PayFast integration documentation for the exact callback validation rules.

4 Verification and Maintenance

Verification checklist

1 Test verified payment, cancellation, failure and a browser return before callback processing. Access must follow server registration state.

2 Send a duplicate COMPLETE notification and concurrent completion requests. Confirm only one payment result and application profile.

3 Reject wrong signature, merchant, reference and amount without activating the account.

4 Simulate Firebase lookup failure and Data Connect failure after payment verification. Confirm reconciliation can recover without another charge.

5 Adapt RegistrationPaymentInitiationTest and PayfastItnRegistrationTest to the Data Connect repositories and add real connector integration coverage. Their later direct-database mocks are not proof that this extension works.

Implementation boundaries

The Firebase baseline GET /api/me automatically creates a profile. A paid-registration extension must revise that behaviour so profile existence cannot bypass the registration gate. Keep status and checkout routes reachable by a verified but not-yet-registered identity.

Private coaching payments need a separate booking reservation and expiry policy. Registration payment code does not implement coaching checkout, fee ownership or refund handling. Define those contracts before enabling paid coaching.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).

Payment flow reference: later repository revision a3cc4c3. The proposed Data Connect entities and operations in this part have not been implemented or tested by this documentation task.

https://developers.payfast.co.za/

https://firebase.google.com/docs/sql-connect/admin-sdk